Equipo docente: Mg. María Alejandra Vranic - Esp. Lic. Gustavo Siciliano - Esp. Lic. Ezequiel Scordamaglia - Lic. Oscar Ruina

Auxiliares docentes: Emiliano Ginarte y Rodrigo Obando

Trabajo Práctico Grupal 2026: Sistema de Gestión "Epicentro Gourmet"

Colaboradores en el TP: Bonino Ezequiel - Mendez Livia - Obando Rodrigo - Ramil Elías - Cofman Santiago - Saucedo Ulises Justo - Savall Arián - Ginarte Delgado Emiliano - Gómez Eric

El centro de convenciones y predio ferial "Epicentro Gourmet" está organizando una serie de festivales temáticos anuales y, para optimizar la administración de estos eventos, requiere un sistema que permita gestionar las unidades de venta, el personal especializado de cada unidad y el rendimiento económico de cada jornada.

De cada festival organizado se debe conocer su nombre, la temporada en la que ocurre y sus fechas de inicio y fin. Para que estos festivales funcionen, se habilitan unidades de venta que se identifican por un nombre comercial, un responsable a cargo (que es uno de los miembros del Staff), la superficie en m² que ocupan y un código único de 10 caracteres que debe contar con una lógica de validación propia.

Existen unidades de tipo Food Truck, que registran su patente y si requieren o no conexión eléctrica para operar, y unidades de tipo Puesto Desarmable, que registran la cantidad de carpas que los componen y el tiempo que toma su montaje en minutos. Cada una de estas unidades debe administrar su propia lista de platos ofrecidos y su lista de personal asignado o staff.

El personal del predio debe ser mayor de edad y se registra con su nombre, apellido, dni, fecha de nacimiento y una fecha de ingreso, la cual es fundamental para determinar su antigüedad.

Todos los empleados poseen un sueldo base, pero existen particularidades según su rol: quienes se desempeñan en la cocina poseen una tarea especial y perciben un plus fijo por categoría. En tanto los cajeros registran su turno de trabajo, ya sea mañana o noche y cobran antigüedad.

Por otro lado, los platos del menú cuentan con un nombre, un precio de venta y un costo de producción. Cada vez que se realiza una operación, se genera un Pedido que debe registrar la fecha de la transacción, el festival donde ocurrió, la unidad de venta que realizó la entrega, los platos y la cantidad pedida de cada plato.

Además, cada festival establece una serie de costos que pueden aplicarse a todas las unidades o algún tipo particular de unidad, los costos son: costo por superficie, costo por montaje, plus por uso de electricidad, sueldo base.

Hito 2: Spring Boot

Desarrollar una aplicación empresarial utilizando el framework de Spring Boot, la aplicación contará con distintas vistas para que el usuario pueda navegar y realizar distintas acciones.

Al ejecutar la aplicación se desplegará una vista (página de inicio) en el que se detalla como mínimo la información de los festivales (datos pre-cargados en la base de datos) y se podrá realizar dos acciones:

1. **Registrar** una **Unidad de Venta** con su **Responsable**, lo que genera un usuario para el responsable (campos obligatorios usuario y contraseña).
2. **Para iniciar sesión**, se debe tener un usuario **Administrador** (usuario precargado en la base de datos) o estar registrado como **Responsable** o como **Empleado**.

Tras el inicio de sesión, el sistema muestra vistas y habilita acciones según el rol del usuario autenticado:

Los **Administradores** pueden realizar las siguientes acciones:

1. **Ver festivales**

2. **Ver unidades de venta de un festival**

3. **Ranking de Unidades:** Para un Festival, generar una lista de objetos ordenada descendente por recaudación.

4. **Unidades con Mayor Canon**: Dado un festival devuelve las “x” unidades (ej. parámetro x=3) que más gastaron en canon indicando nombre comercial, código, tipo de unidad y el canon. (usar clase `ReporteMayoresCanonDTO, no persiste.`

Los **Responsables** pueden realizar altas de **empleados** y **platos,** y consultar reportes varios sobre la unidad de venta que tiene asignada:

1. **Altas, Bajas y Modificaciones:** Métodos para agregar, modificar y eliminar empleados. Cuando se da de alta un empleado se deberá **enviar un mail** al empleado con el usuario (su email) y una contraseña generada aleatoriamente para que pueda iniciar sesión en la aplicación. La eliminación de los empleados será de tipo lógica. En caso de que se quiera dar de alta una entidad eliminada utilizar un método de actualización específico.

2. Filtros **por un atributo o varios atributos (tener en cuenta de traer entidades que no estén eliminadas lógicamente)**

3. **Cálculo de Canon:** Método que devuelve el monto a pagar por una unidad:
    - *Food Truck:* (areaSup * $500) + $2000 si requiere conexión eléctrica.
    - *Puesto:* (areaSup * $500) - (Tiempo de Montaje * $10).

4. **Liquidación de Haberes:** Calcular el sueldo del personal según su tipo:
    - Sueldo Base: $100.000
    - 1 año de antigüedad $ 5000.-

Rol en la cocina	**Plus por categoría**
Cocinero	$ 150.000.-
Cocinero ayudante	$ 100.000.-
Lavaplatos	$ 50.000.-

Cantidad de días que dura el Evento	**Rol en la cocina** (Sueldo base + Plus por categoría)	Cajero (Sueldo Base + antigüedad)
Hasta 15 días	50% (sueldo base + plus por categoría)	50% (Sueldo Base + antigüedad)
De 16 a 30	100% (sueldo base + plus por categoría)	100% (Sueldo Base + antigüedad)
Más de 30 días	suma proporcional según intervalos anteriores (1)	suma proporcional según intervalos anteriores (2)

(1) totalAcobrarCocina=(int(diasDurFest/15)+1) * 50% *(sueldoBase+plusCategoria)

por ej: festival= 38 días, rol=lavaplatos
totalAcobrarCocinero=(int(38/15)+1) * 50% *($100.000+$50.000) = $225.000

(2) totalAcobrarCajero=(int(diasDurFest/15)+1) * 50% *(sueldoBase+antigüedad)

por ej: festival= 52 días, años antigüedad=3 años
totalAcobrarCajero=(int(52/15)+1) * 50% *($100.000+ 3* $5.000) = $230.000

5. **Reporte de Recaudación:** Dado un festival, retornar su recaudación total (usar clase `ReporteVentaDTO`, no persistente).

6. **Cálculo de Rentabilidad Neta:** Calcular la ganancia de una Unidad en un Festival (Pedidos totales - Costos de platos) y restar obligatoriamente los sueldos y el canon. Los pedidos deben estar pre-cargados en la base de datos.

7. **Cálculo de Rentabilidad Neta entre dos fechas**: Para un Festival y

una unidad, calcular la rentabilidad neta entre dos fechas.

8. **Plato Estrella:** Dado una Unidad y un Festival, devolver el objeto `PlatoDTO` que registró mayor cantidad de Pedidos.

Los **empleados** pueden realizar las siguientes acciones:

1. Ver su información:
    - Festivales, Unidades de Venta en las que trabajaron.

2. Sueldo recibido en cada festival (el empleado puede **descargar un recibo de sueldo en formato PDF**).

3. Ver en la actualidad si tiene que trabajar en un festival activo y en qué unidad.

4. Cajero:
    - Cerrar Caja al finalizar el día

Generar los siguientes flujos para realizar todas las acciones requeridas, estos son:

1. Login
2. Registro de Responsable con Unidad de Venta.
3. Alta de Empleados.
4. Alta de Platos.
5. Reportes tanto de administradores como responsables.
6. Ver Festival y Puesto de Venta actual en el que se trabaja.
7. Descargar recibo de sueldo.

Notas: los flujos 2 a 4 corresponden a el Responsable, los flujos 6 y 7 a Empleados.

Ejemplo de flujo de alta de empleados:
- Ingreso a la página de inicio.
- Iniciar sesión como usuario Responsable.
- Completar un formulario con los datos necesarios.
- Dar de alta un empleado, enviar un mail al empleado dado de alta.
- Ver todos los empleados activos.

Notas a tener en cuenta para resolver el Sistema:

1. Iterar el **Diagrama de clases para la Gestión de un Festival**.
2. Listar los casos de uso principales.
3. Implementar casos de uso que sean reales para la gestión y análisis de ventas, costos y ganancias de festivales o unidad de venta de un festival.
4. Tener en cuenta en la funcionalidad no utilizar datos “hardcodeados” en los métodos, sino pensar en alguna clase para encapsular esos datos como por ej. sueldo base que es $100.000.

Tecnologías a utilizar:

IDEs recomendados: **Spring Tool Suite** - **Intellij IDEA** - **Visual Studio Code**.
Manejador de dependencias: **Maven**
Base de datos: **MySQL**.
Lenguaje: **JAVA**
Framework: **Spring Boot**.

Calendario de Defensa y Entrega:

Semana	T. Mañana	T. Noche	Corrección
12	28/10	29/10	Defensa Hito 2
13	04/11	05/11	Defensa Hito 2

Github y Metodología de Trabajo:

1. Detallar en el Readme:
    - Número de Grupo.
    - Integrantes y usuario de GitHub.
    - Pasos para correr la aplicación. (crear bd, variables de entorno, etc).
    - Enlace a Diagrama de clases
    - **Casos de uso implementados y responsables.**
2. Manejo del Repositorio:
    - Cada caso de uso se debe agregar a la rama main mediante **Pull Request.**
    - Generar una rama con el nombre **CUFuncionalidadResponsable**, por ejemplo **CUAltaEmpleadoJuanPerez.** Desarrollar el caso de uso y crear el Pull Request correspondiente.
    - En el PR detallar en comentarios lo realizado (creación de entidades, repositorios, servicios, etc).
    - **Colocar en el Readme el link al PR** en la sección de caso de uso implementado y responsable.

Parámetros de evaluación:

Diagrama de clases.
Listado de Casos de uso
Nivel de Solución del Sistema
Buenas prácticas
BD.
Readme.
Vistas y navegación entre las mismas.
Implementación Casos de Uso.
Seguridad.
Envío de mails.
Uso de Git (aportes al Proyecto).
Uso de variables de entorno.

Trabajo en equipo

Cada equipo debe tener en el desarrollo del Hito 2:
Por grupo:

- por lo menos un envío de e-mail
- por lo menos un DTO de reporte
- por lo menos una generación de un pdf

Por Integrante: 2 CU mas de los enunciados que no sean los grupo